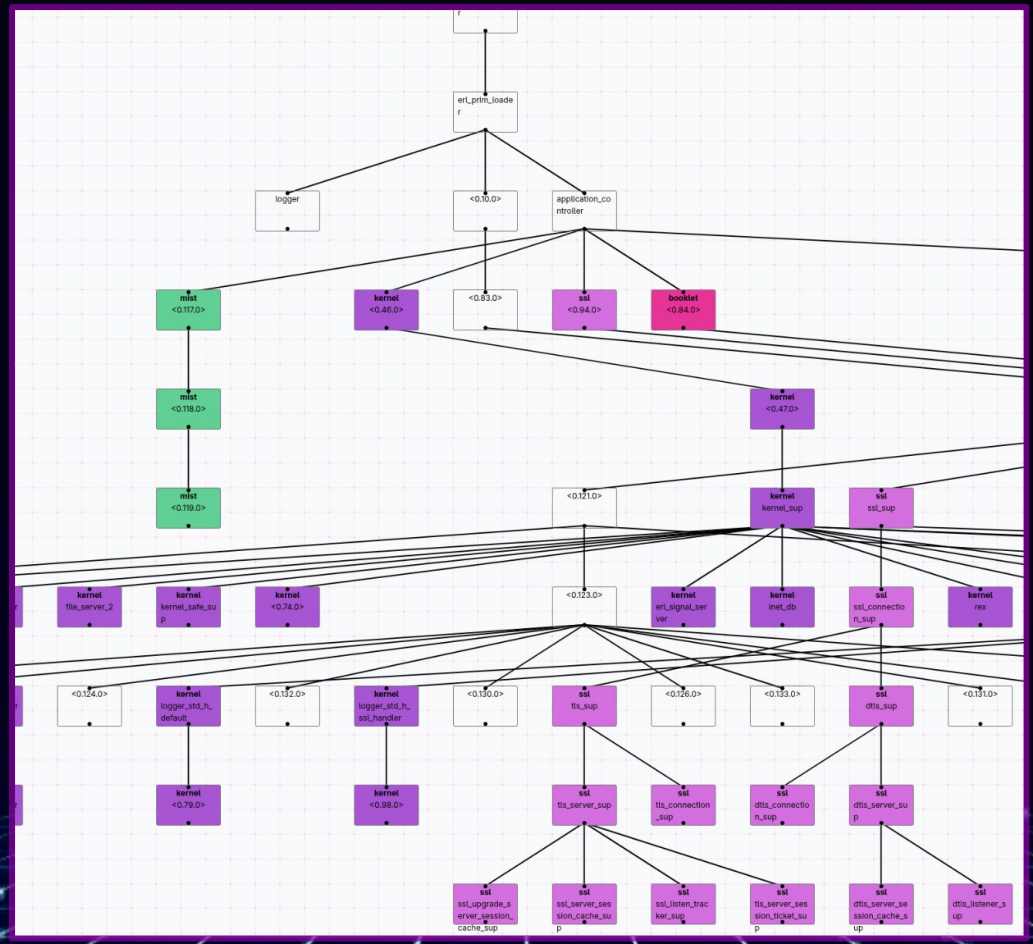
ASTERISM

A *Journey* to the Heart of the BEAM

> START <

ASTERISM


erl_prim_loade
r
logger
<0.10.0>
application_co
ntroller
mist
<0.117.0>
kernel
<0.46.0>
<0.83.0>
ssl
<0.94.0>
booklet
<0.84.0>
mist
<0.118.0>
kernel
<0.47.0>
mist
<0.119.0>
<0.121.0>
kernel
kernel_sup
ssl
ssl_sup
kernel
file_server_2
kernel
kernel_safe_su
p
kernel
<0.74.0>
<0.123.0>
kernel
erl_signal_ser
ver
kernel
inet_db
ssl
ssl_connectio
n_sup
kernel
rex
<0.124.0>
kernel
logger_std_h_
default
<0.132.0>
kernel
logger_std_h_
ssl_handler
<0.130.0>
ssl
tls_sup
<0.126.0>
<0.133.0>
ssl
dtls_sup
<0.131.0>
kernel
<0.79.0>
kernel
<0.98.0>
ssl
tls_server_sup
ssl
tls_connection
_sup
ssl
dtls_connectio
n_sup
ssl
dtls_server_su
p
ssl
ssl_upgrade_s
erver_session_
cache_sup
ssl
ssl_server_ses
sion_cache_su
p
ssl
ssl_listen_trac
ker_sup
ssl
tls_server_ses
sion_ticket_su
p
ssl
dtls_server_se
ssion_cache_s
up
ssl
dtls_listener_s
up

Guillaume Heu



Cisco instructor

DevOps/SysAdmin

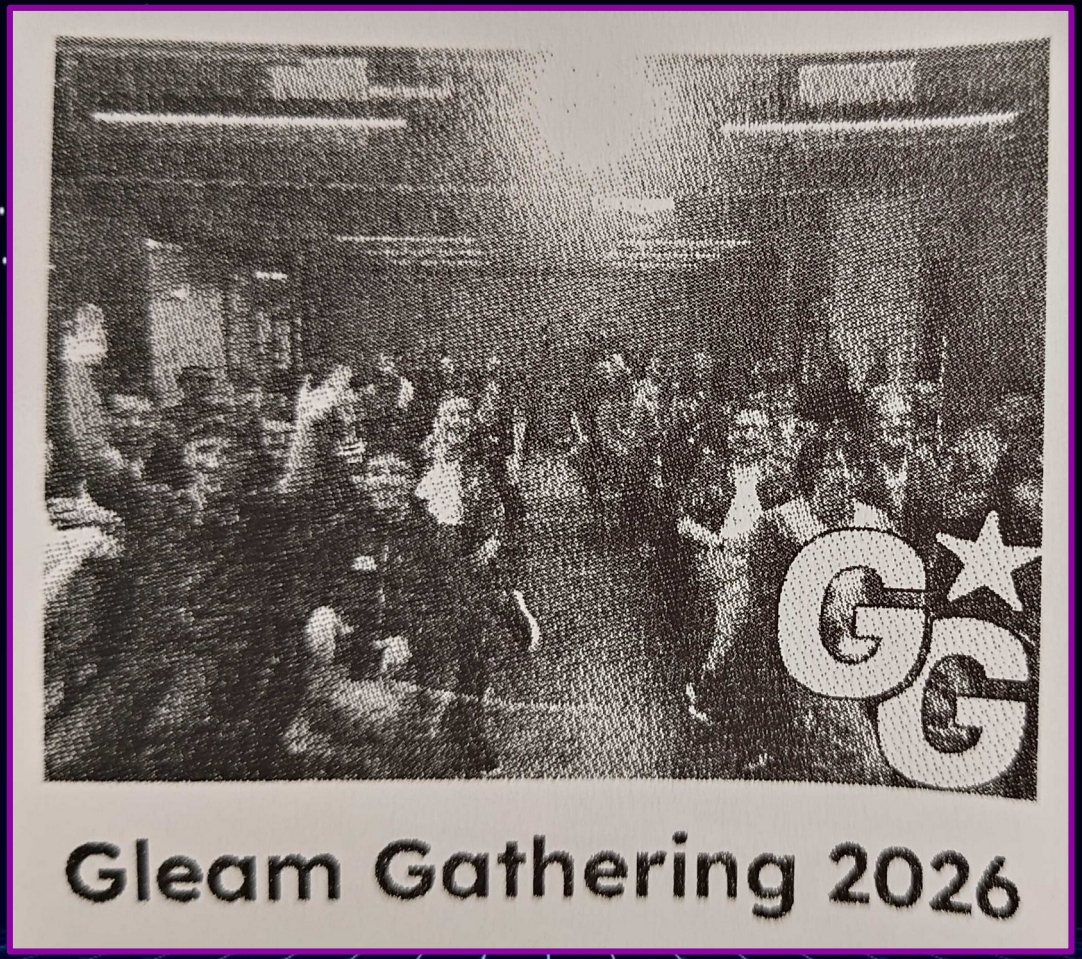
Recovering Nix-aholic

Avid Gleamlin!

guillheu.dev

blog.guillheu.dev

github.com/guillheu


GG
Gleam Gathering 2026

Goals

- Represent a BEAM node's process tree
- Identify supervision relationships
- Highlight applications
- Animate messages being passed between processes
- ~~Collapse~~ supervision trees

Part 1
OTP is not BEAM

Part 1
OTP is not BEAM

A more accurate formulation: *ERTS (containing the BEAM VM) provides the concurrency and fault-isolation primitives. OTP is the distribution that ships ERTS together with standard libraries and a framework of behaviours and design principles built on those primitives.*

– Claude

The BEAM/OTP Iceberg

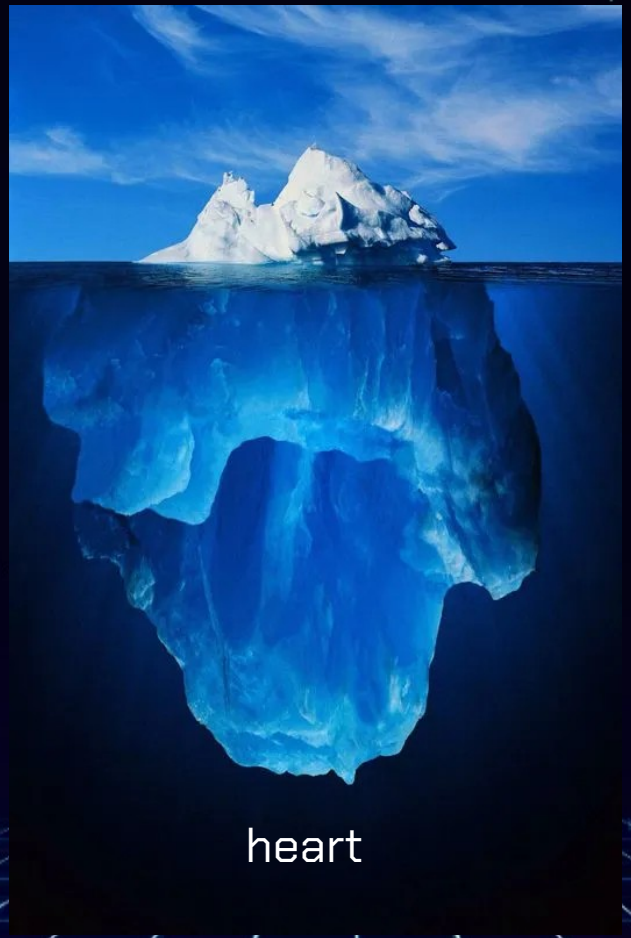
processes

workers

applications

trap_exit flag

gen_statem



links

supervisors

GenServer / Gleam actors

group leader

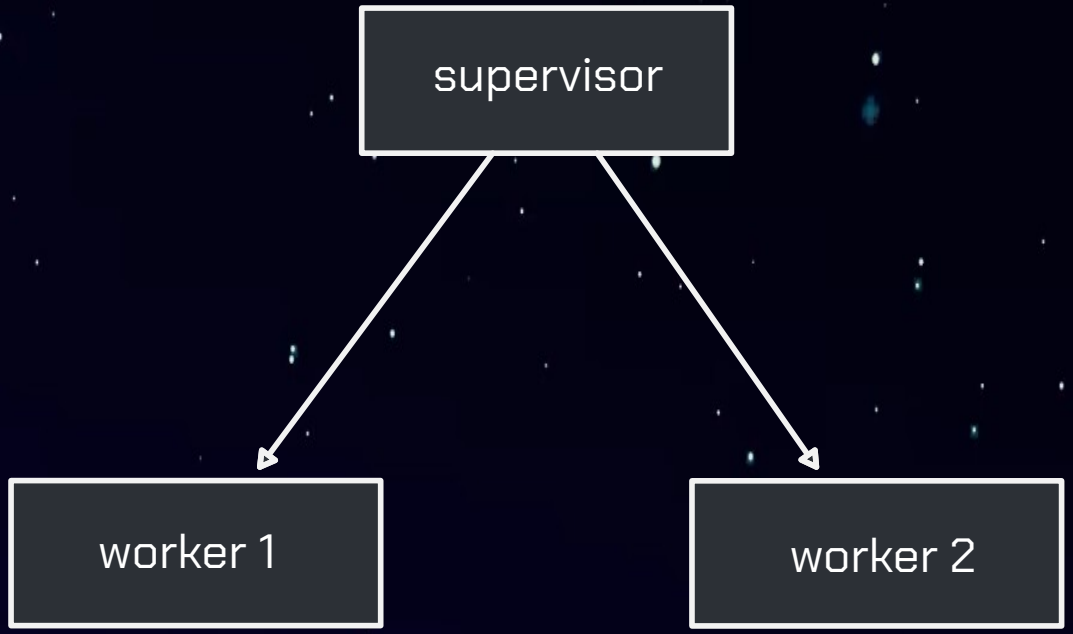
kernel

Represent a BEAM node's process tree

Not so fast...

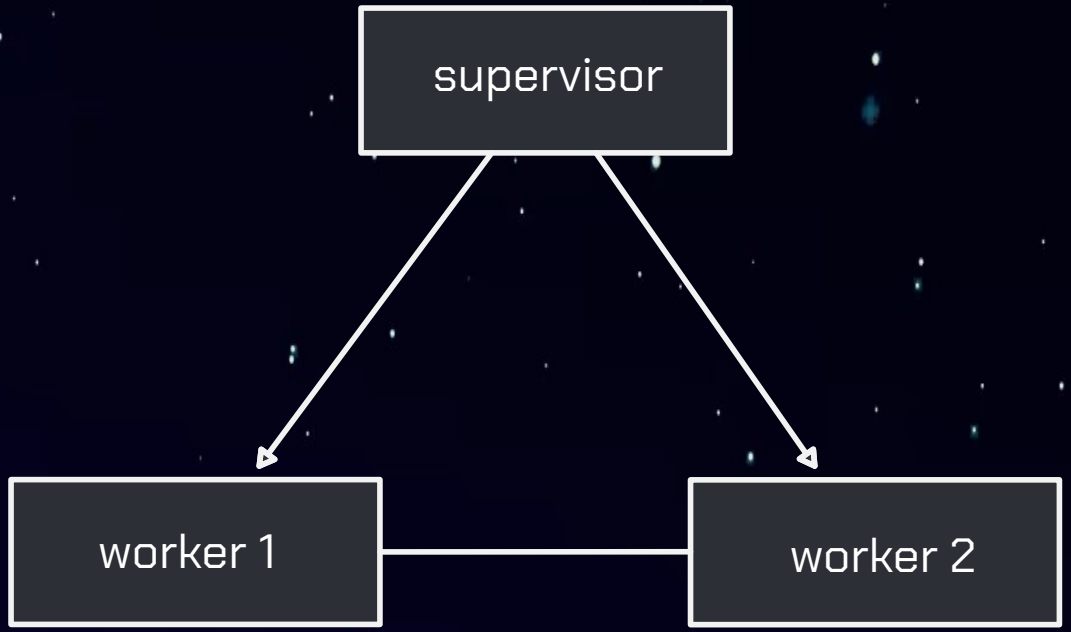
Process Graph

Is a tree?



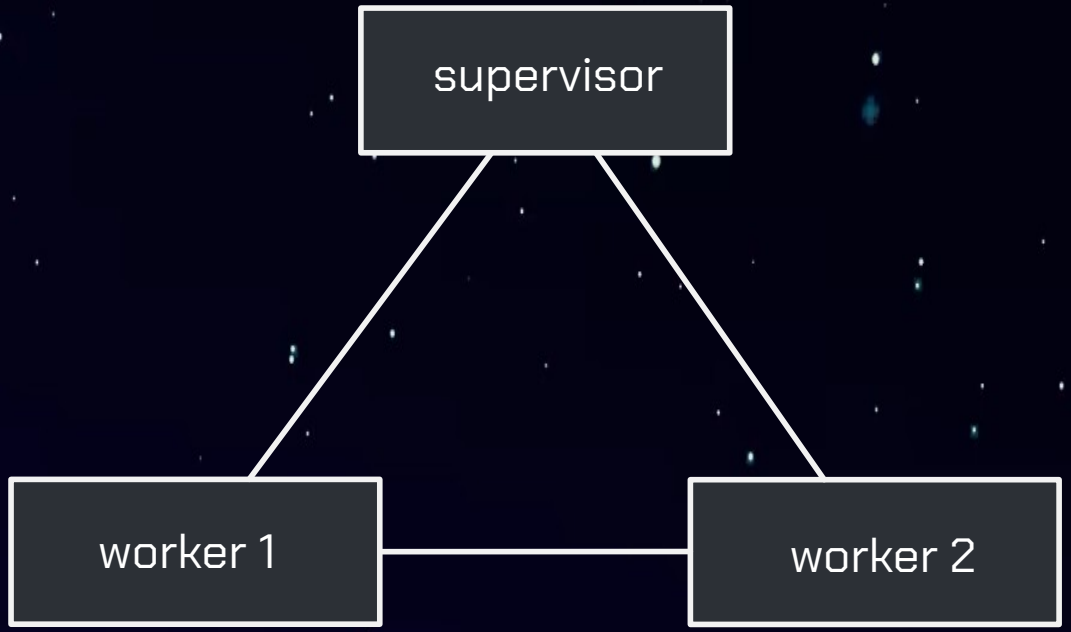
Process Graph

Not a tree!



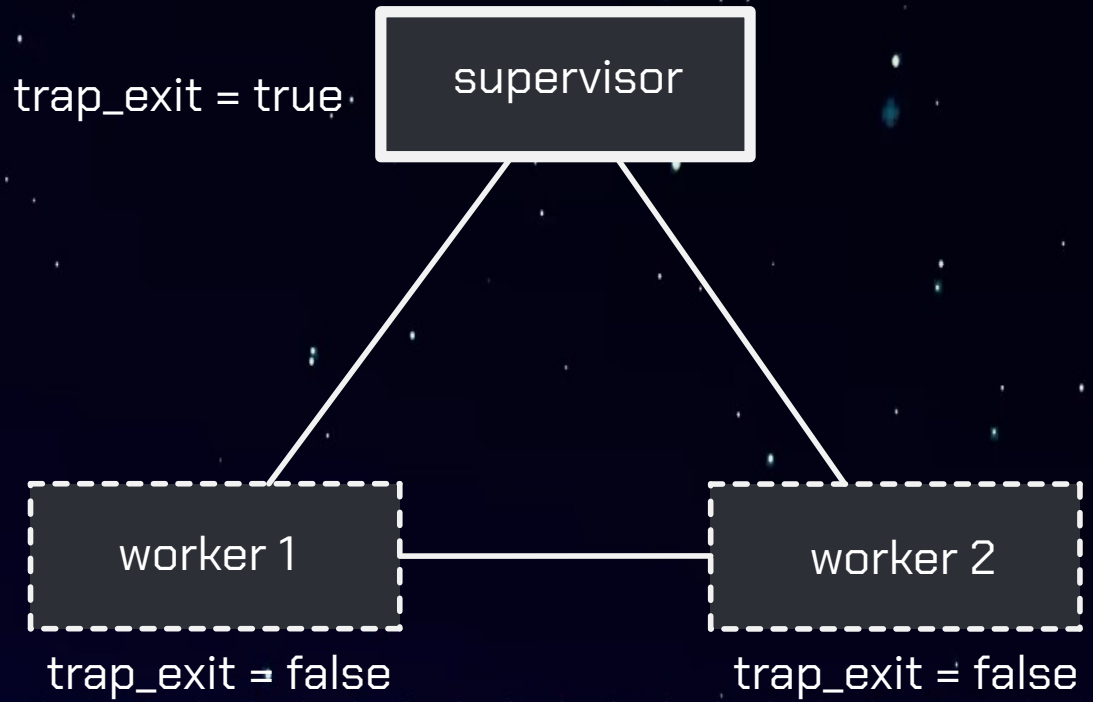
Process Graph

Not even directed!



Process Graph

Not even a real supervisor!!!



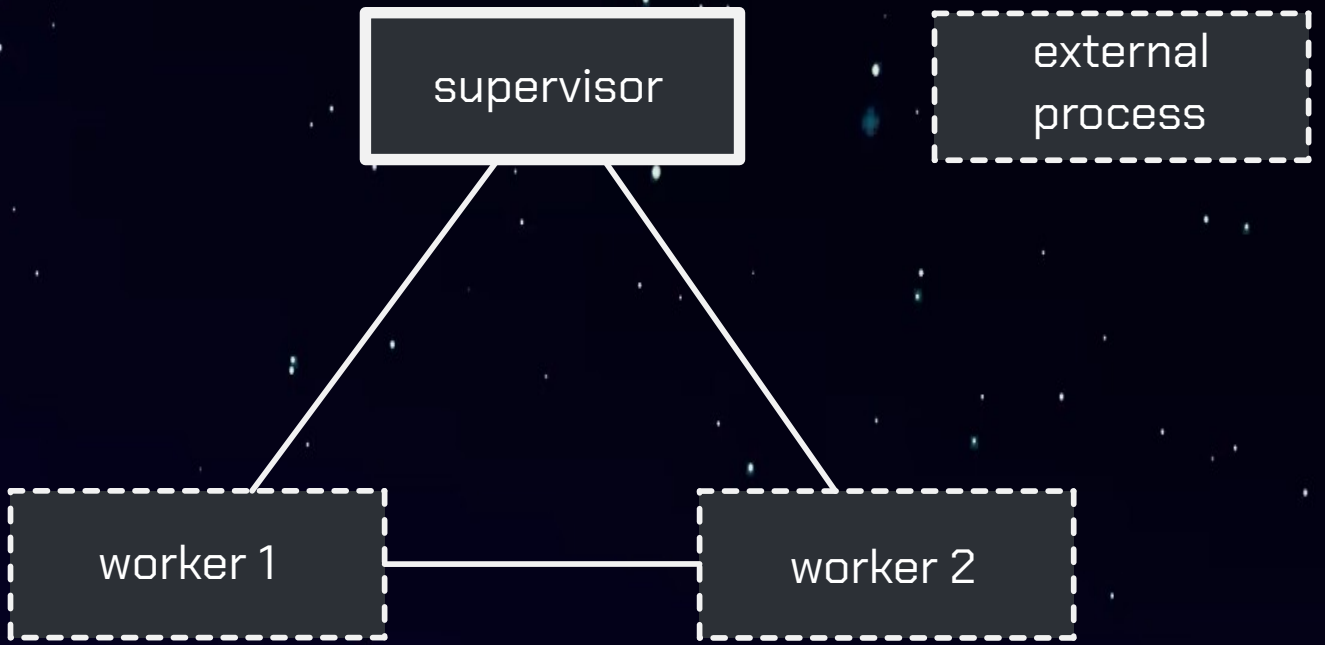
https://github.com/erlang/otp/blob/OTP-29.1/lib/stdlib/src/supervisor.erl#L950

```
949     init({SupName, Mod, Args}) ->
950         process_flag(trap_exit, true),
951         case Mod:init(Args) of
952             {ok, {SupFlags, StartSpec}} ->
953                 case init_state(SupName, SupFlags, Mod, Args) of
954                     {ok, State} when ?is_simple(State) ->
955                         init_dynamic(State, StartSpec);
956                     {ok, State} ->
957                         init_children(State, StartSpec);
958                     Error ->
959                         {stop, {supervisor_data, Error}}
960                 end;
961             ignore ->
962                 ignore;
963             Error ->
964                 {stop, {bad_return, {Mod, init, Error}}}
965         end.
```

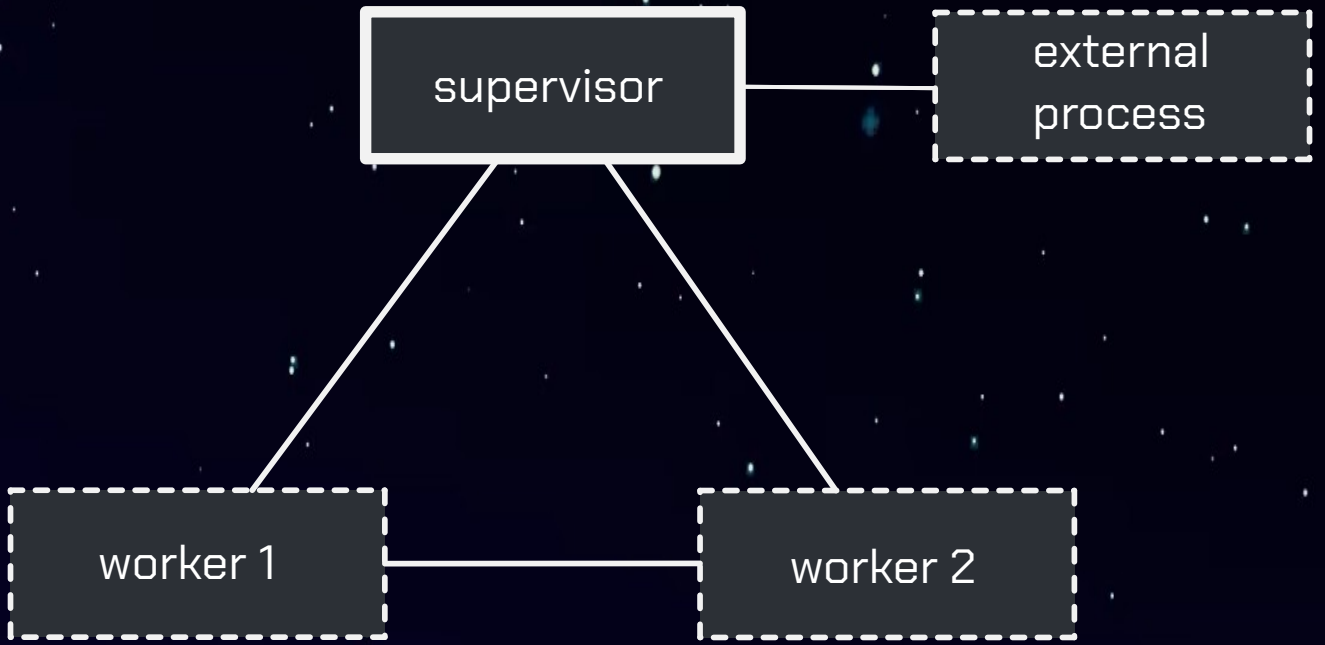
https://github.com/erlang/otp/blob/OTP-29.1/lib/stdlib/src/supervisor.erl#L1287

```
1278    -spec handle_info(term(), state()) ->
1279            {'noreply', state(), gen_server:action()} | {'stop', 'shutdown', state()}.
1280
1281    handle_info({hibernate, Tag}, #state{tag = Tag} = State) ->
1282        {noreply, enter_hibernation(State), hibernate};
1283
1284    handle_info(Msg, #state{hibernating = true} = State) ->
1285        handle_info(Msg, wakeup(State));
1286
1287    handle_info({'EXIT', Pid, Reason}, State) ->
1288        case restart_child(Pid, Reason, State) of
1289            {ok, State1} ->
1290                {noreply, State1, hibernate_after_action(State)};
1291            {shutdown, State1} ->
1292                {stop, shutdown, State1}
1293        end;
1294
1295    handle_info(Msg, State) ->
1296        ?LOG_ERROR("Supervisor received unexpected message: ~tp~n",[Msg],
1297                   #{domain=>[otp],
1298                     error_logger=>#{tag=>error}}),
1299        {noreply, State, hibernate_after_action(State)}.
1300
```

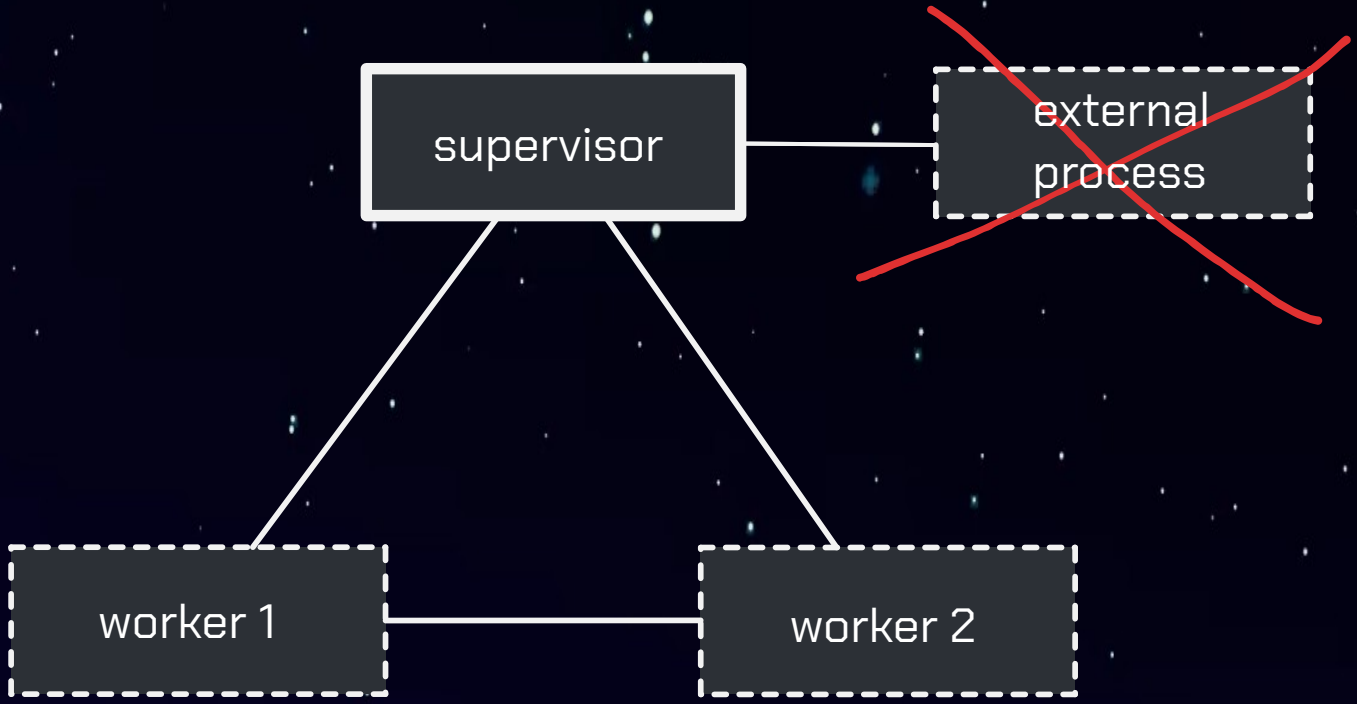
Supervision



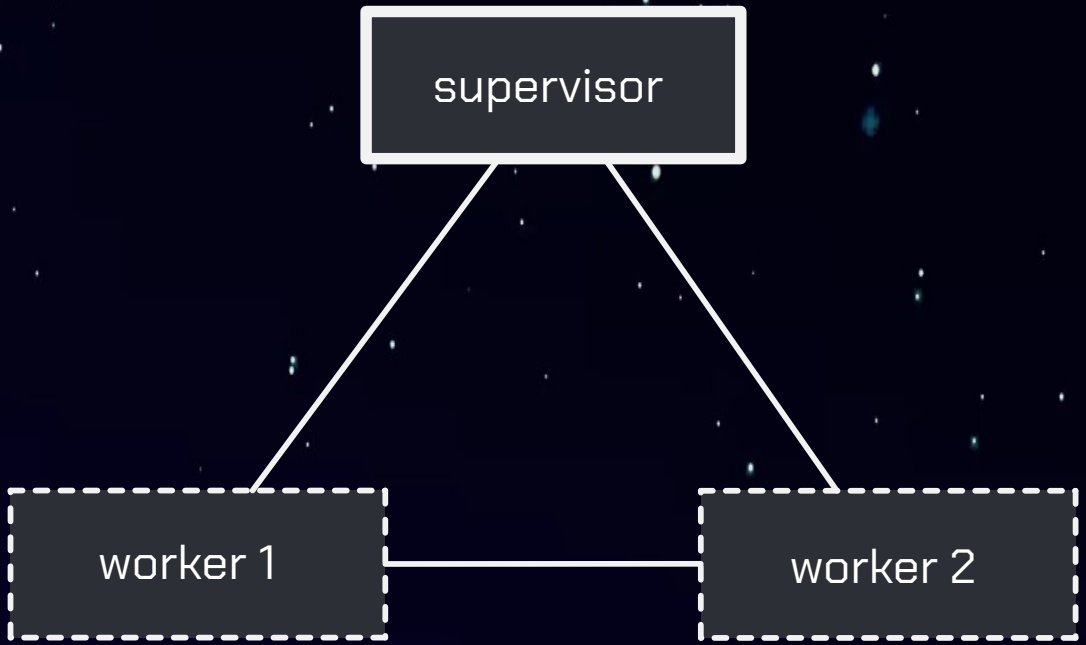
Supervision



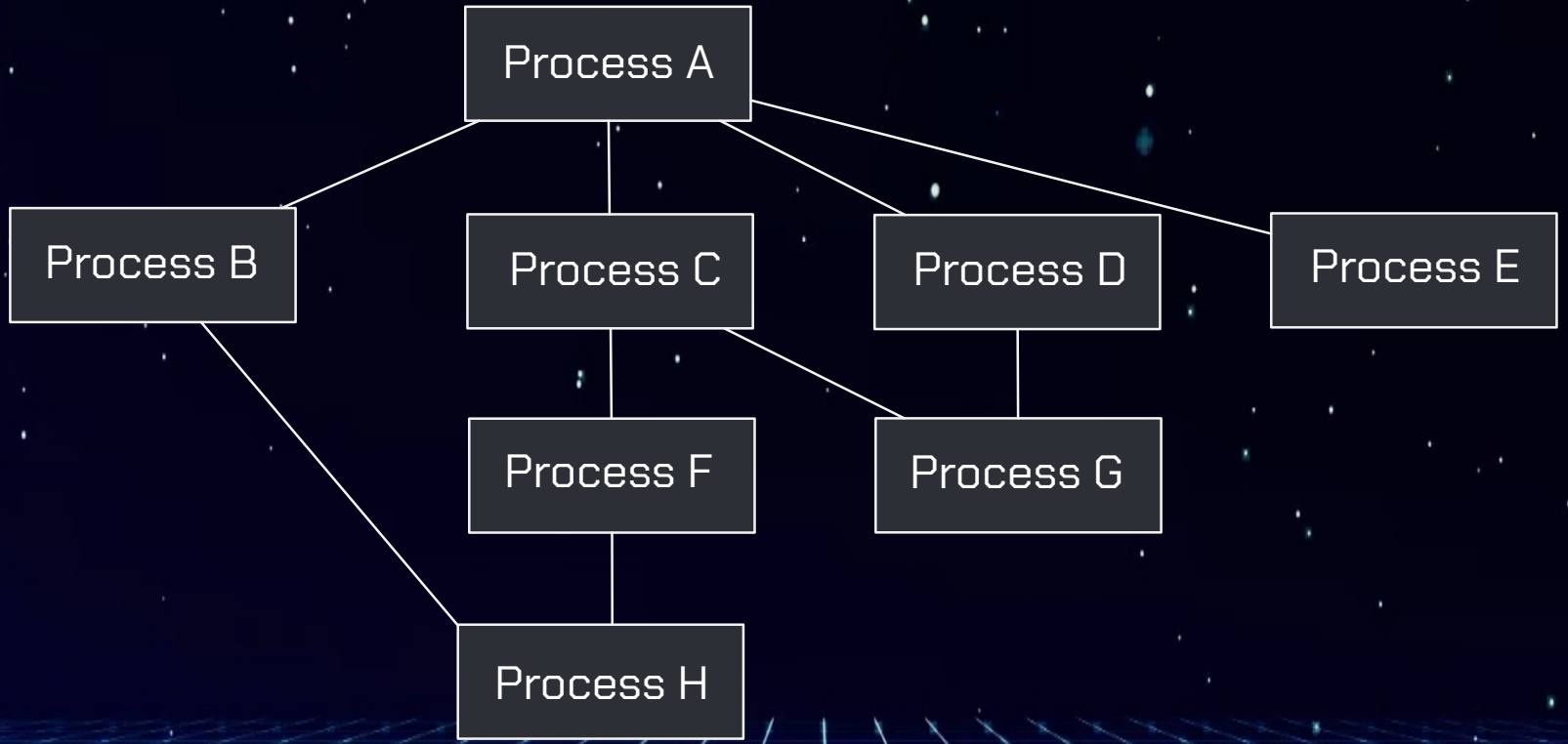
Supervision



Supervision



Process forest



Goals

- ~~Represent a BEAM node's process tree~~
- Identify supervision relationships
- Highlight applications
- Animate messages being passed between processes
- ~~Collapse~~ supervision trees

What's a supervisor?

What's a supervisor?

- Process (can receive messages)

What's a supervisor?

- Process (can receive messages)
- trap_exit = true

What's a supervisor?

- Process (can receive messages)
- trap_exit = true
- Keeps a state of its children

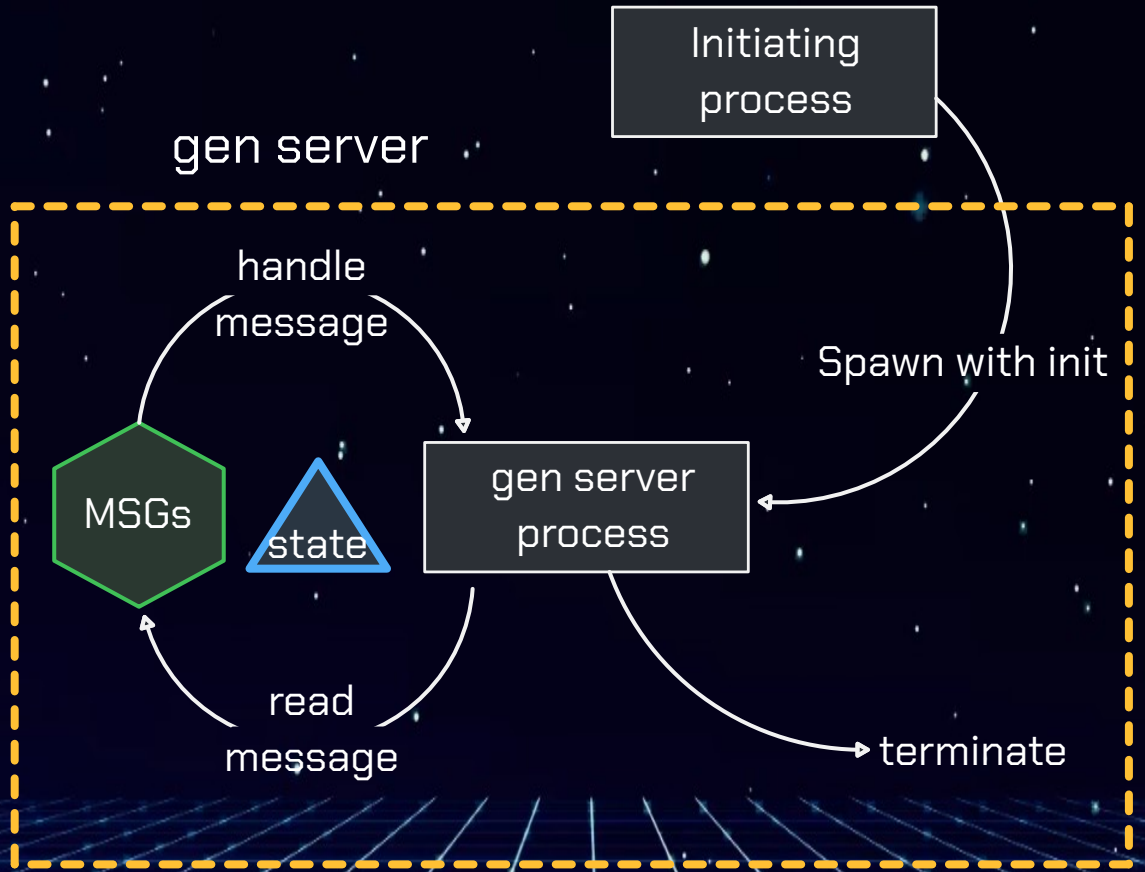
What's a supervisor?

- Process (can receive messages)
- trap_exit = true
- Keeps a state of its children
- Has a restart strategy

What's a supervisor?

- Process (can receive messages)
- trap_exit = true
- Keeps a state of its children
- Has a restart strategy
- Is a gen server

What's a gen server?



What's a gen server?



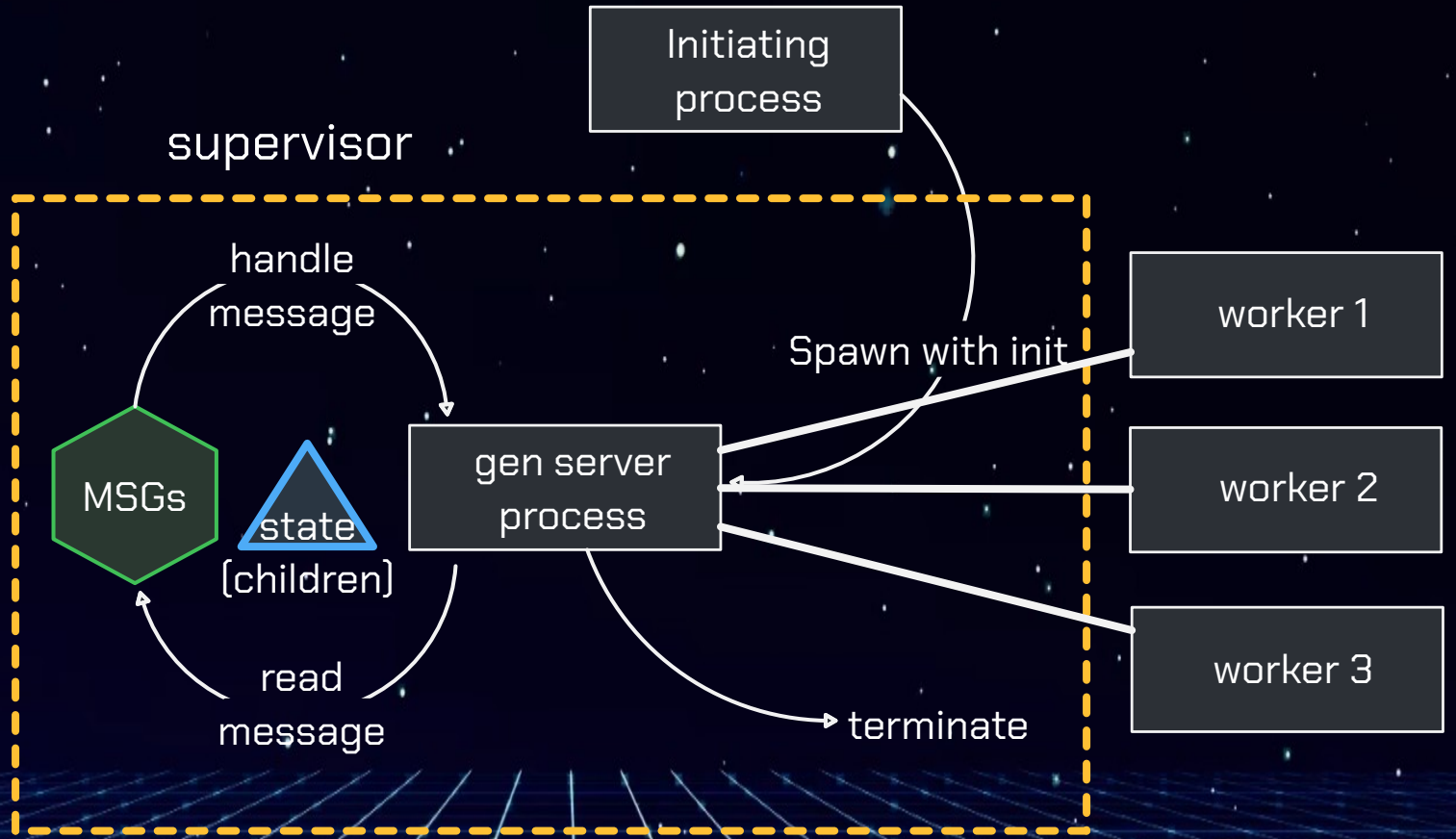
What's a gen server?



What's a gen server?



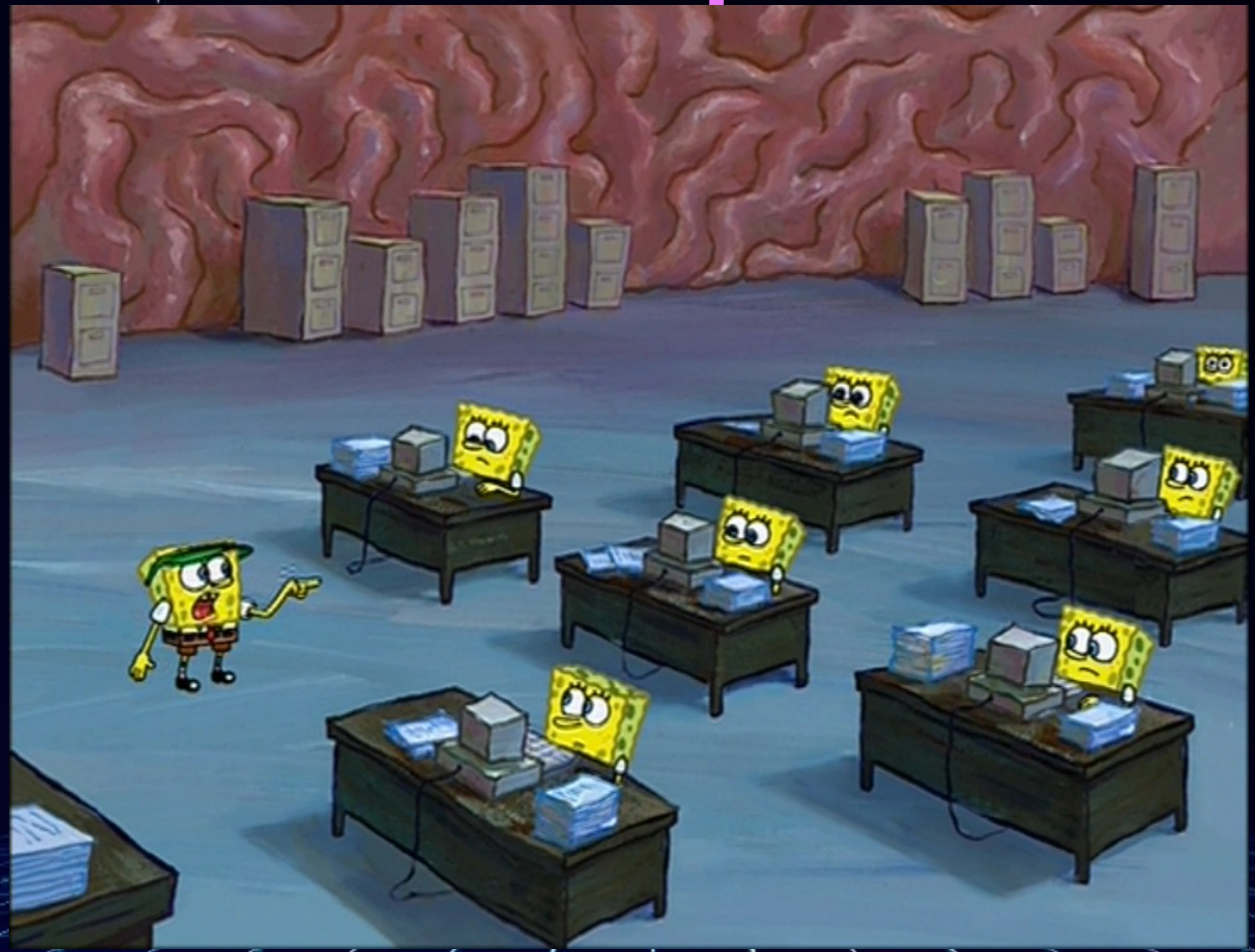
What's a supervisor?



What's a supervisor?



What's a supervisor?



Identifying a supervisor

Just send it a message!

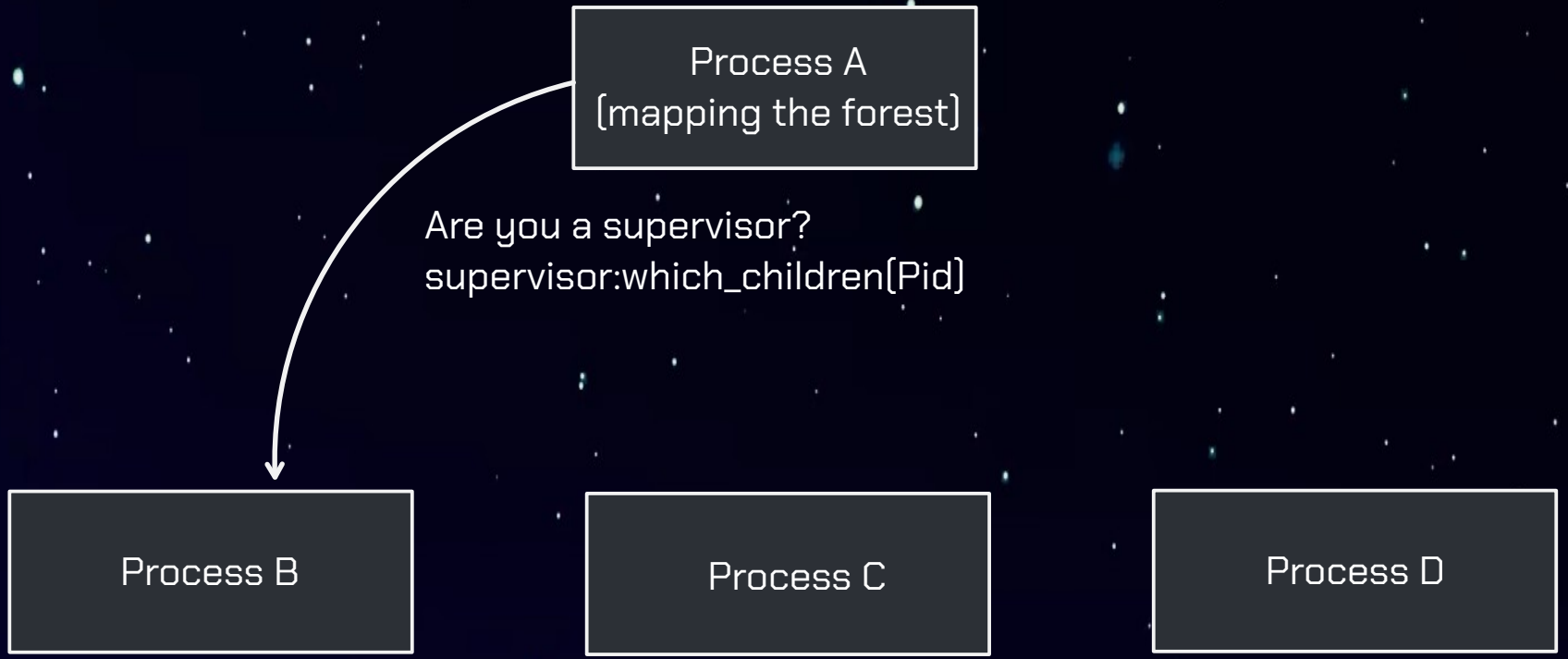
```
1169    handle_call(which_children, _From, State) ->
1170        Resp =
1171            children_to_list(
1172              fun(Id,#child{pid = ?restarting(_),
1173                            child_type = ChildType, modules = Mods}) ->
1174                    {Id, restarting, ChildType, Mods};
1175                 (Id,#child{pid = Pid,
1176                            child_type = ChildType, modules = Mods}) ->
1177                    {Id, Pid, ChildType, Mods}
1178              end,
1179              State#state.children),
1180        {reply, Resp, State, hibernate_after_action(State)};
```

I ran into a problem...

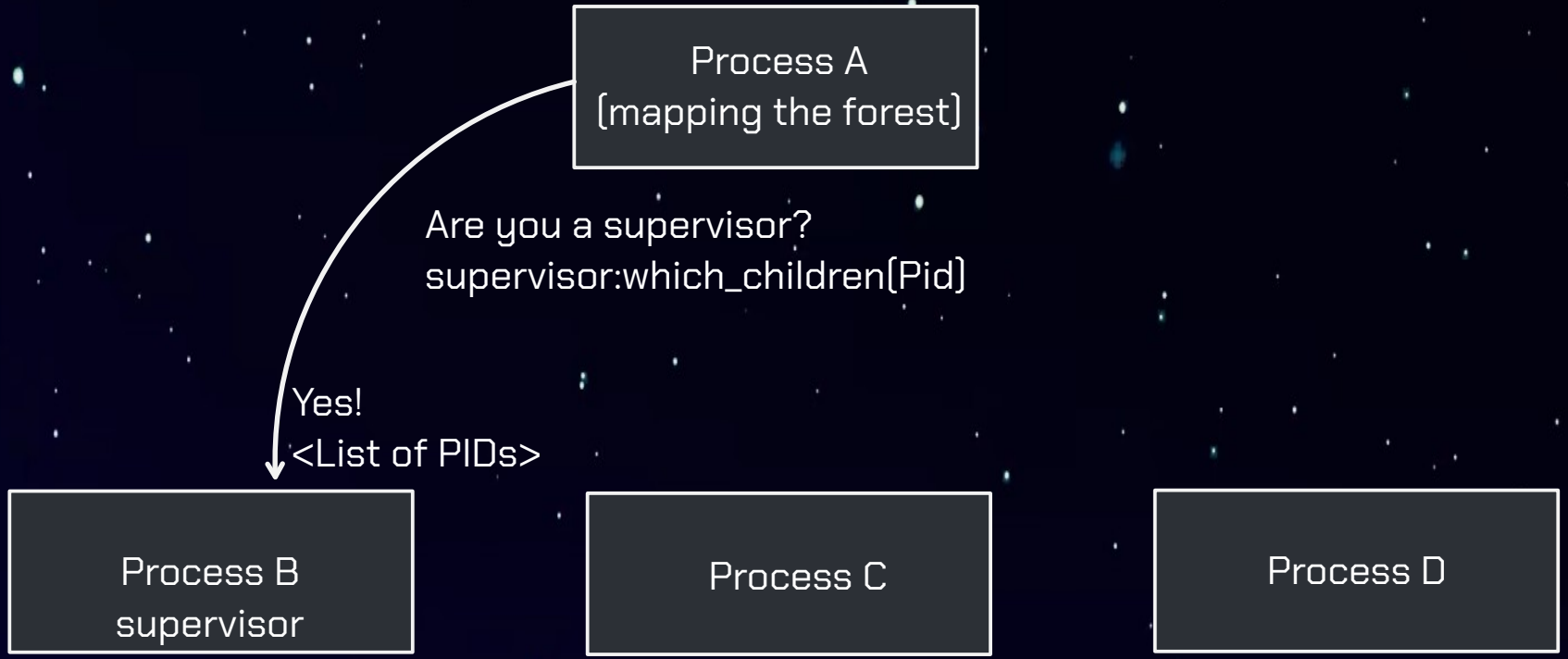
Identifying a supervisor



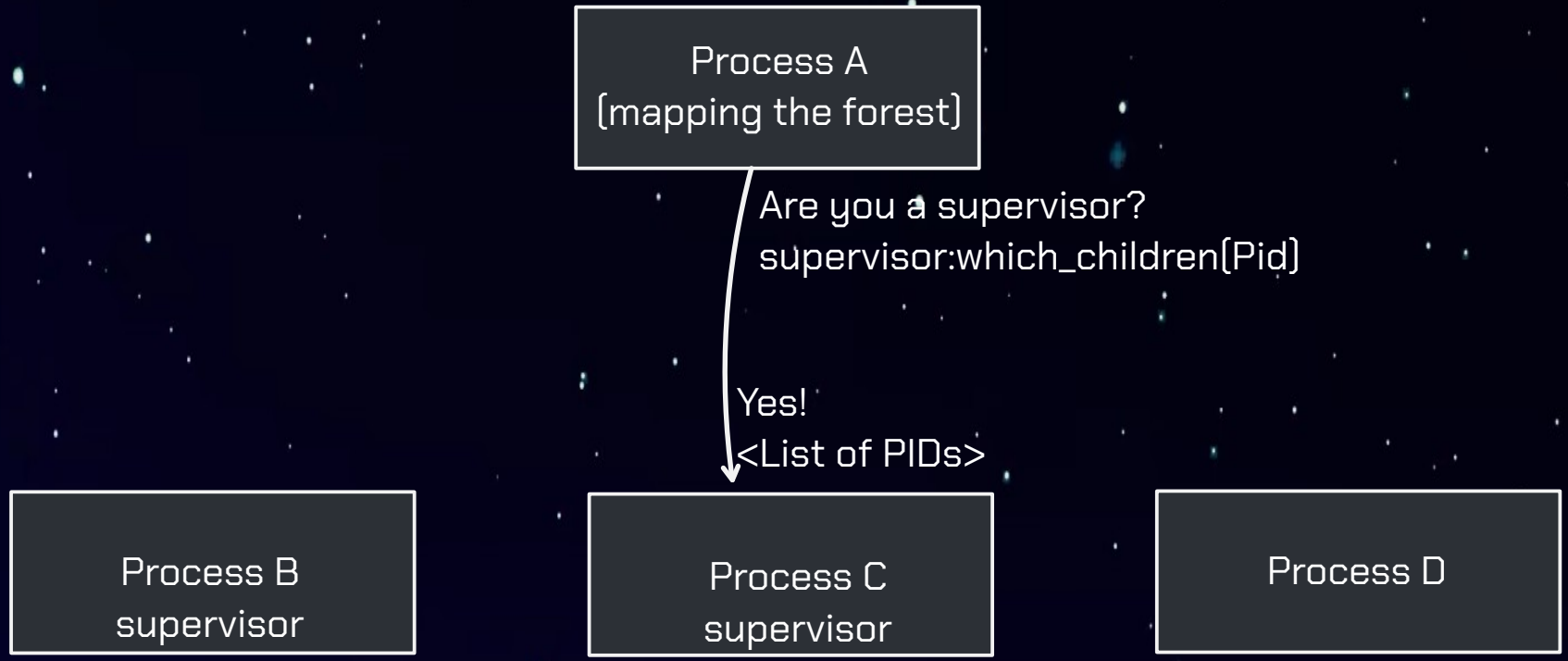
Identifying a supervisor



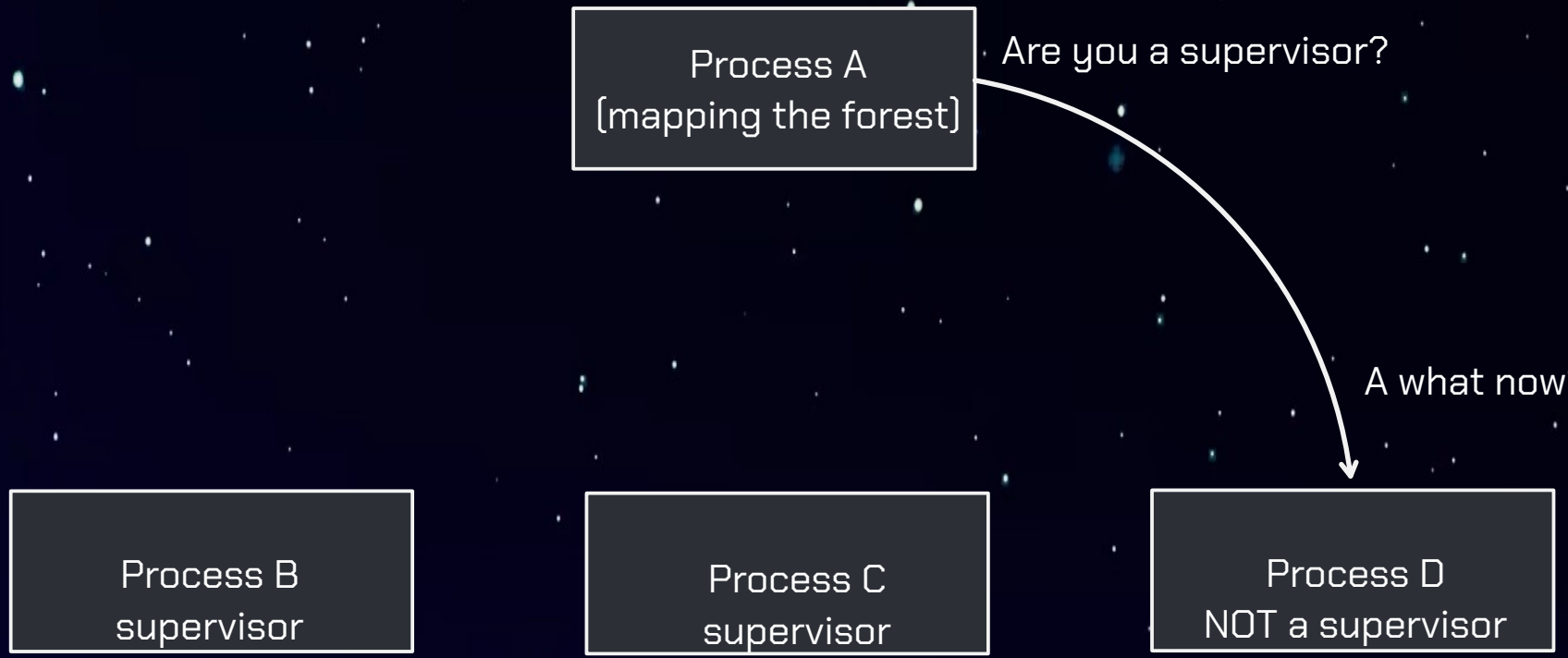
Identifying a supervisor



Identifying a supervisor



Identifying a supervisor



Identifying a supervisor



Identifying a supervisor



You've got mail!

Process A
(mapping the forest)

Am I a supervisor?

Process B
supervisor

Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor



You've got mail!

Process A
(mapping the forest)

Am I a supervisor?

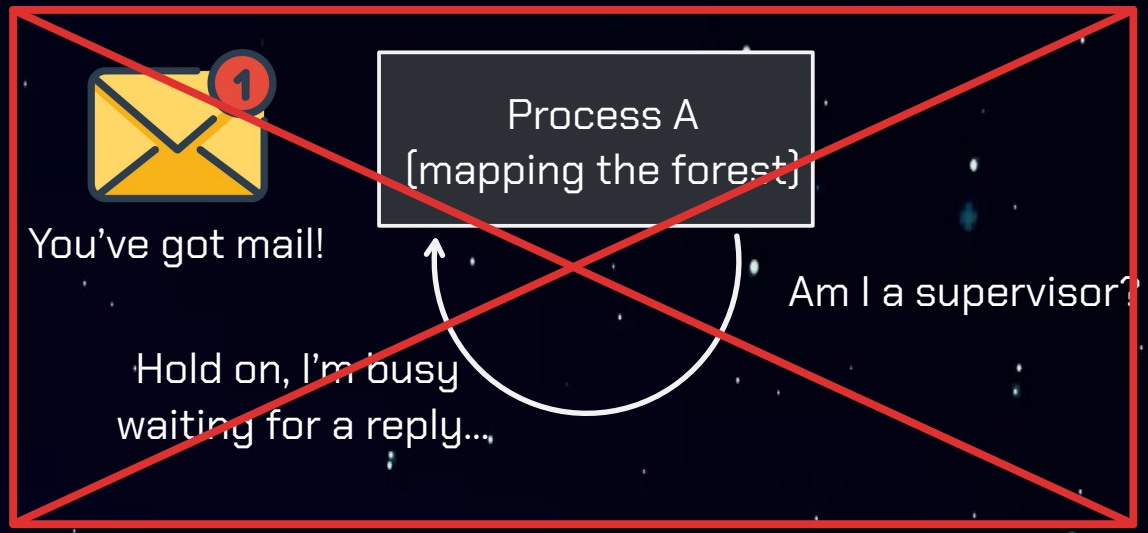
Hold on, I'm busy
waiting for a reply...

Process B
supervisor

Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor



Process B
supervisor

Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor



Identifying a supervisor

Process A
(mapping the forest)



Process B
supervisor

Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor

Process A
(mapping the forest)



Does this process even look like a supervisor?

Process B
supervisor

Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor

Process A
(mapping the forest)



Does this process even look like a supervisor?

```
process_info(Pid, {dictionary, '$initial_call'})
    = {_, {supervisor, _Mod, 1}}
```

Process B
supervisor

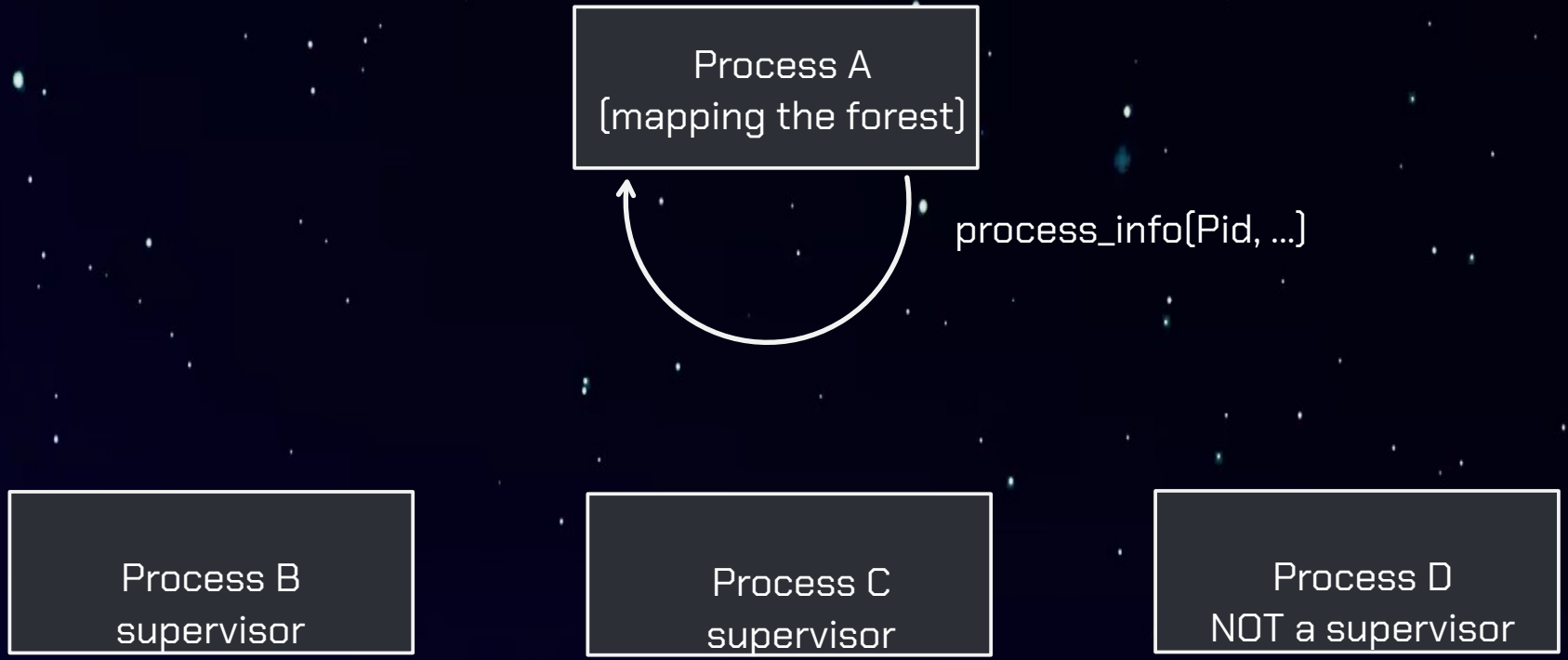
Process C
supervisor

Process D
NOT a supervisor

Identifying a supervisor



Identifying a supervisor



Goals

- ~~Represent a BEAM node's process tree~~
- ~~Identify supervision relationships~~
- Highlight applitations
- Animate messages being passed between processes
- ~~Collapse~~ supervision trees

Applications

asterism.app (generated)

```
{application, asterism, [
   {vsn, "1.0.0"},
   {applications, [aonyx_graph,
                   clique,
                   [...]
                   mist,
                   sugiyama]},
   {description, ""},
   {modules, [asterism,
              asterism@internal@lustre@app,
               [...]
              asterism_test]},
   {registered, []}
]}.
```

Applications

asterism_ffi.erl

```
get_process_application(Pid) when node(Pid) =:= node() ->
    case application:get_application(Pid) of
        {ok, App} -> {ok, App};
        undefined -> {error, nil}
    end;
```

Applications

application.erl is in kernel

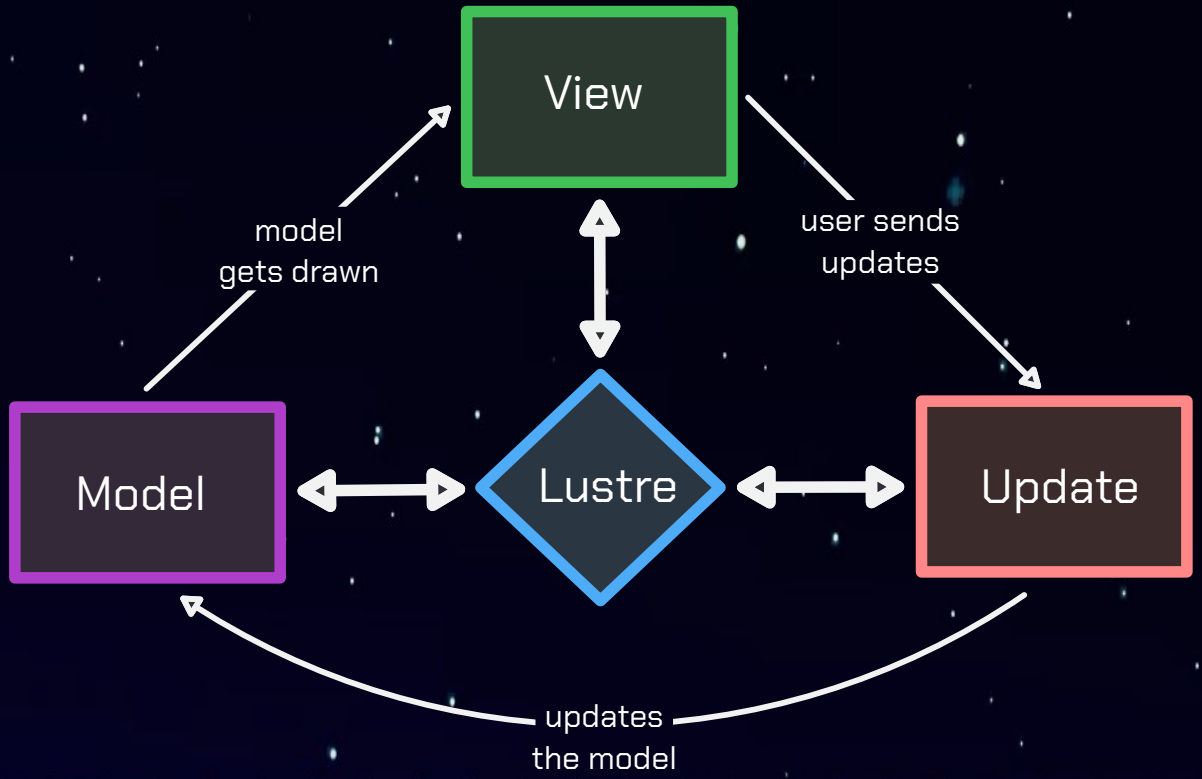
Not in stdlib

Goals

- ~~Represent a BEAM node's process tree~~
- ~~Identify supervision relationships~~
- ~~Highlight applitations~~
- Animate messages being passed between processes
- ~~Collapse~~ supervision trees

Part 2
Putting it all on canvas

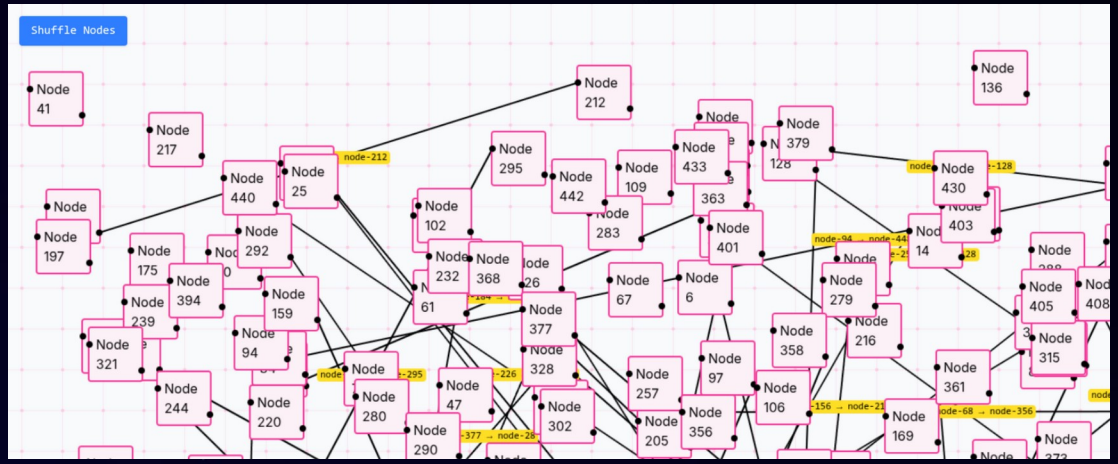
Lustre


View
model
gets drawn
user sends
updates
Model
Lustre
Update
updates
the model

Lustre

and Clique

https://github.com/**hayleigh-dot-dev**/**clique**

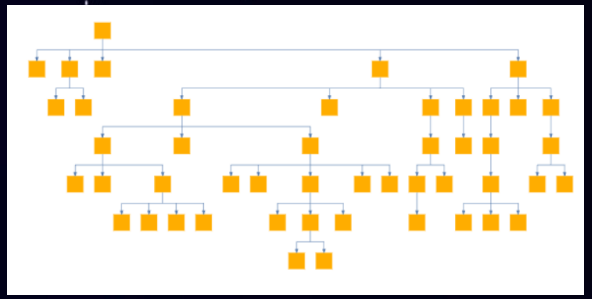


Caveats:

- No layout algorithm
- Not published
- Still a WIP

Layouts

Is mayonnaise a layout?



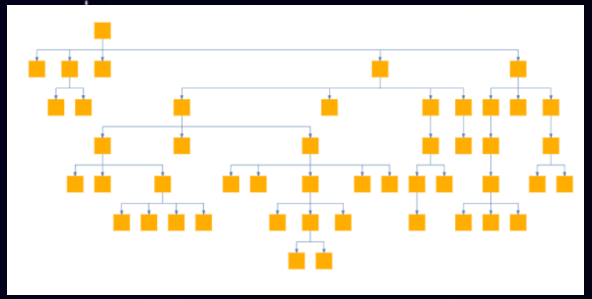
Tree

Layouts

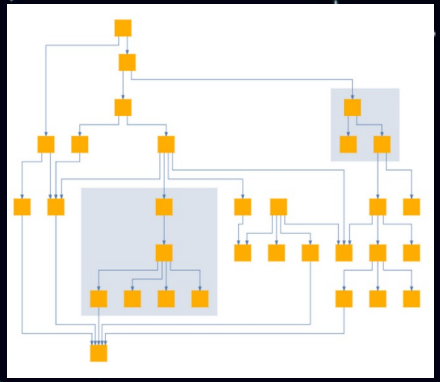
Is mayonnaise a layout?

Layouts

Is mayonnaise a layout?



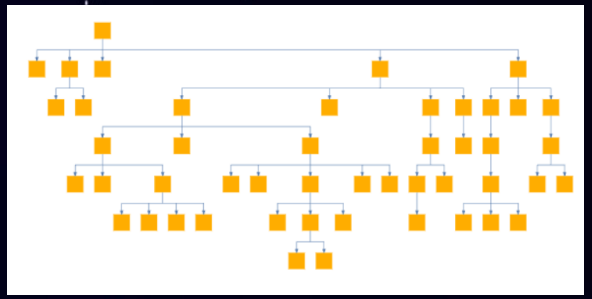
Tree



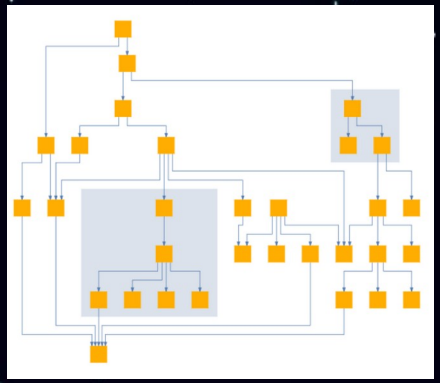
Hierarchical

Layouts

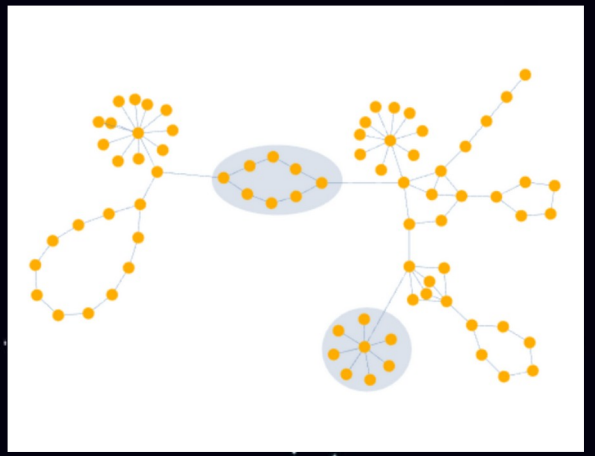
Is mayonnaise a layout?



Tree



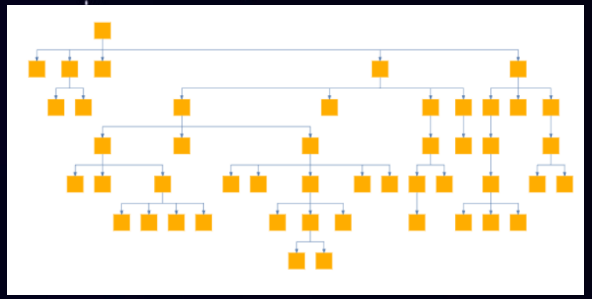
Hierarchical



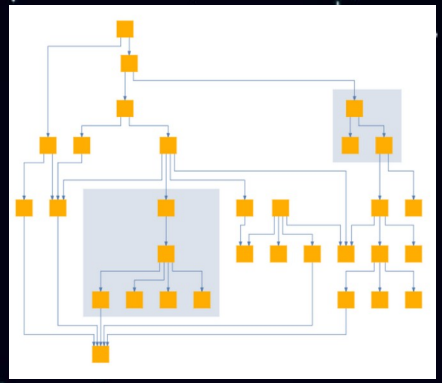
Organic (force-directed)

Layouts

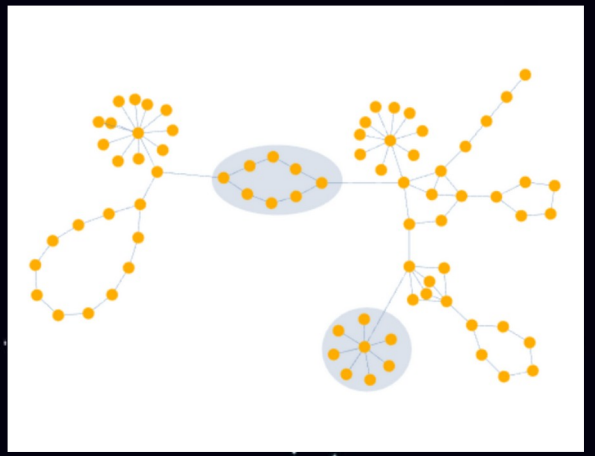
Is mayonnaise a layout?



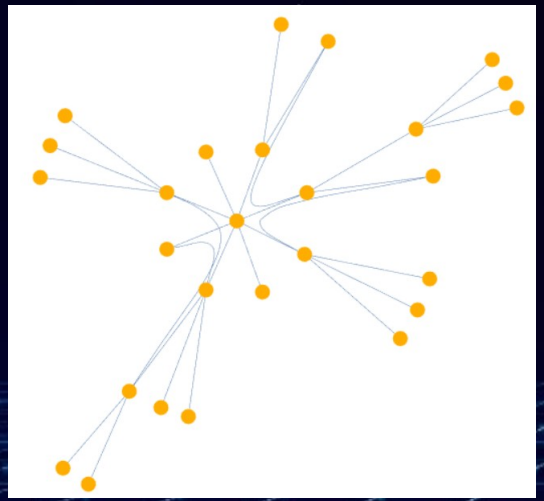
Tree



Hierarchical



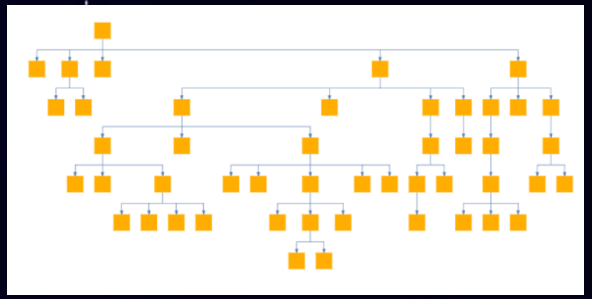
Organic (force-directed)



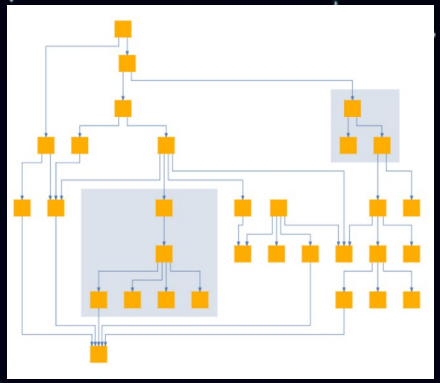
Radial

Layouts

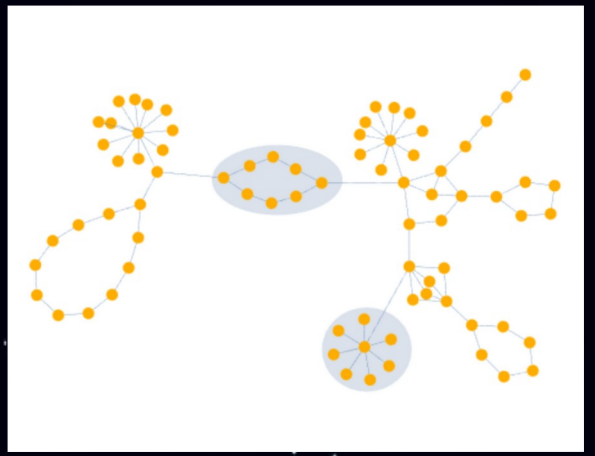
Is mayonnaise a layout?



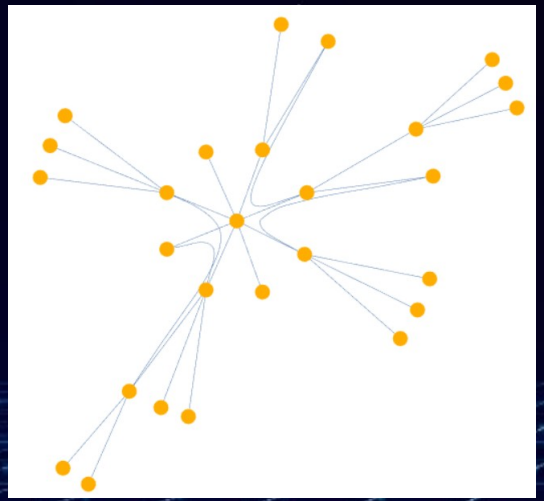
Tree



Hierarchical

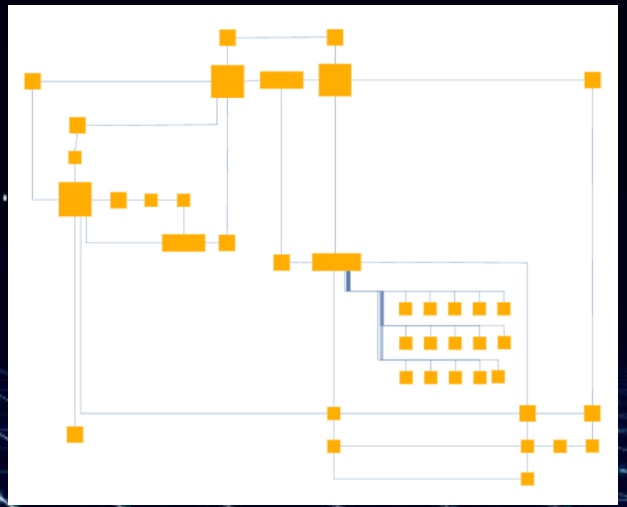


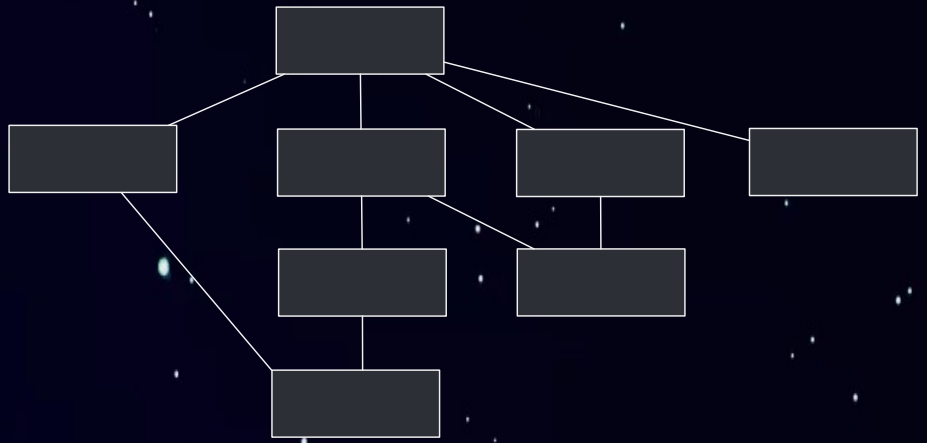
Organic (force-directed)



Radial

Orthogonal







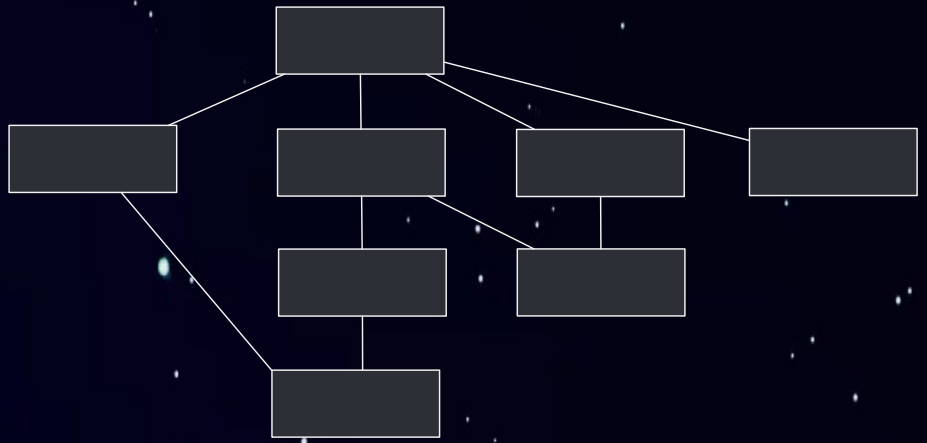
Compute

Render











Compute & render







Wheel

163 languages

Article Talk | Read View source View history

From Wikipedia, the free encyclopedia

For other uses, see Wheel (disambiguation) and Wheels (disambiguation).

A **wheel** is a rotating component (typically circular in shape) intended to turn on an axle bearing. The wheel is one of the key components of the wheel and axle, which is one of the six simple machines. Wheels, in conjunction with axles, allow heavy objects to be moved easily, facilitating transportation, supporting loads, and performing work in machines. Wheels are also used for other purposes, such as a ship's wheel, steering wheel, potter's wheel, and flywheel.

Common examples can be found in transport applications. A wheel reduces friction by facilitating motion by rolling together with the use of axles. For a wheel to rotate, a moment must be applied to the wheel about its axis, either by gravity or by the application of another external force or torque.



Spoked wheel of reconstructed Celtic funerary wagon, 550 BCE, Hochdorf Chieftain's Grave, Germany

Type	Invention
Inventor	First recorded use in Sumer (modern-day south-central Iraq) Guillaume Heu (co-inventor)
Inception	Before 3500–3350 BCE
Available	Globally

Layouts

Hierarchical/Sugiyama



That's what I call "Caving"!



Layouts

Hierarchical/Sugiyama

- Re-implementing solved problem
- Big time gain (my head hurts)
- Prototype

Part 3
Quick demo!

What's next?

- Show isolated processes
- Display supervision relationships (edges)
- Collapse applications/supervision trees

Feedback wanted!

Links

github.com/guillheu/asterism



github.com/guillheu/sugiyama-gleam

github.com/haileigh-dot-dev/clique



blog.guillheu.dev/
lustre-mist-and-beam-messages

Links

Gleam Gathering

2027

GOOD PEOPLE. GLEAM CONVERSATIONS.

SCAN FOR YOUR TICKET

LucyIsMyHero

Part 4

This is the part where I thank you and answer questions before getting off stage